



**FACULTY OF COMPUTER SCIENCE AND MATHEMATICS
BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH
HONOURS**

**CSE3433 SOFTWARE ARCHITECTURE
SEMESTER II SESSION 25/26**

PROJECT TITLE:

**TALENTFLOW – EVENT-DRIVEN JOB RECRUITMENT & APPLICATION
PROCESSING SYSTEM**

PREPARED FOR:

Dr. Mohamad Nor Bin Hassan

PREPARED BY:

NAME	MATRIC NO
IZZ DHIA DAMIA BINTI MAT ZAIDI	S75849
NURUL IFFAH SABRINA BINTI AHMAD SHUKRI	S75894

TABLE OF CONTENT

Introduction.....	2
System Overview.....	2-3
Problem Description	3
System Objectives.....	4
Selected Architecture Style.....	4
Functional Requirement.....	5
Non-functional Requirement.....	6
Architecture Style Explanation	6-7
Service or Component Identification.....	7
Architecture Diagrams.....	8-12
Team Roles.....	12

Introduction

TalentFlow is an innovative recruitment management system that aims to bring automation and efficiency to the hiring process. Most conventional recruiting software systems remain highly dependent on manual operations, such as resume verification, updating candidate status, and scheduling interviews. These manual procedures are cumbersome, time-consuming, and challenging to execute, especially in cases involving many applicants.

In order to address these challenges, TalentFlow incorporates the concept of Event-Driven Architecture (EDA). EDA enables the software system to respond instantly to significant events like posting jobs, receiving applications, and updating statuses.

System Overview

TalentFlow is an event-driven recruitment automation solution that aims to streamline the hiring process for organizations. The tool takes care of posting job openings, candidate applications, resumes, interviews, and notifications.

The tool has two primary categories of users:

Candidates

They can look at job opportunities, apply online, get confirmation emails, and receive timely information about their application status.

Recruiters

They can post job descriptions, evaluate candidate applications, update recruitment status, shortlist candidates, and track recruitment progress using dashboards.

Event-Driven Architecture (EDA) powers the software. This means each significant action produces an event. Events automatically initiate corresponding services like screening, notifications, and reporting without waiting for any manual input.

Events in the system include:

- Posting a job vacancy
- Submitting an application
- Screening resumes
- Shortlisting candidates
- Scheduling interviews
- Updating status

This design enables TalentFlow to work effectively, react promptly, and scale up easily while handling several applicants simultaneously.

Problem Description

These days, there are many recruitment processes which remain manual and this results in the following challenges:

- Slow recruitment process - The Human Resource (HR) department has to examine individual resumes and make contacts with candidates.
- No immediate action - If someone applies for any job position, then there will be no immediate action taken on their resume.
- Poor communication - The applicants will have to wait for a while before they get some news about their applications and this may cause the loss of interest from their side.
- High error rate - Manual processing may result in making errors like overlooking some details provided by candidates.
- Too much workload for HR - In case of an overwhelming number of candidates applying for different positions.

System Objectives

The major objectives of TalentFlow include:

- Automation of the recruitment process through automatic actions when an event occurs.
- Reducing the workload of HR by automating resume screening and processing applications.
- Enhancing communication with candidates via instant messaging.
- Avoiding mistakes made by humans with a structured workflow system.
- Speeding up the hiring process with automatic shortlisting and scheduling interviews.
- Offering real-time reporting and statistics.

Selected Architecture Style

Event-Driven Architecture (EDA)

TalentFlow implements **Event-Driven Architecture (EDA)** using a publish-subscribe communication model. The system does not wait for direct requests. Instead, it responds automatically whenever an event occurs inside the platform.

For example:

- Candidate submits application
- Recruiter updates status
- Interview is scheduled
- Resume screening completed

These actions create events that trigger related services immediately.

Functional Requirements

ID	Requirement
FR01	Recruiters can create job posts that automatically update the job board.
FR02	When a candidate submits an application, the system sends a confirmation automatically.
FR03	When recruiters update candidate status, the candidate is notified immediately.
FR04	The system filters resumes automatically based on skills and experience.
FR05	Recruiters can manage candidates in one place (shortlisted, interviewed, rejected).
FR06	The system schedules interviews and sends invitations automatically
FR07	A dashboard displays real-time recruitment progress and statistics.

Non-Functional Requirements

Attribute	Requirement
Scalability	Support thousands of job applications efficiently.
Reliability	No event loss during system load or temporary failures.
Performance	Fast processing and instant notifications.
Availability	Core services available most of the time.
Maintainability	Easy to update or add new modules.

Architecture Style Explanation

The chosen architectural design for this system is that of Event-driven architecture that employs the Publish/subscribe model. TalentFlow works with the help of event-driven communication, which allows its components to communicate with one another rather than using request/response as the sole way of working.

Characteristics used:

- Asynchronous processing – No time delay between different services.

- Loosely coupled components – If resume filtering fails, it doesn't hinder posting jobs.
- Event Channels – RabbitMQ and Kafka are used in this system to form an event bus.
- Instantaneous response – The system allows users to get notifications instantly.

Service or Component Identification

Service	Responsibility	Events Produced	Event Consumed
Job	Manage job post	JobPosted	-
Application	Receive applications	ApplicationSubmitted	JobPosted
Resume Filtering	Auto-screen resumes	ResumeScreened	ApplicationSubmitted
Status Management	Update candidate status	StatusUpdated	ResumeScreened
Notification	Send emails/SMS	NotificationSent	StatusUpdated, InterviewScheduled
Interview	Schedule interviews	InterviewScheduled	StatusUpdated (shortlisted)
Dashboard	Real-time stats	-	All major events (read-only projections)

Architecture Diagrams

System Context Diagram

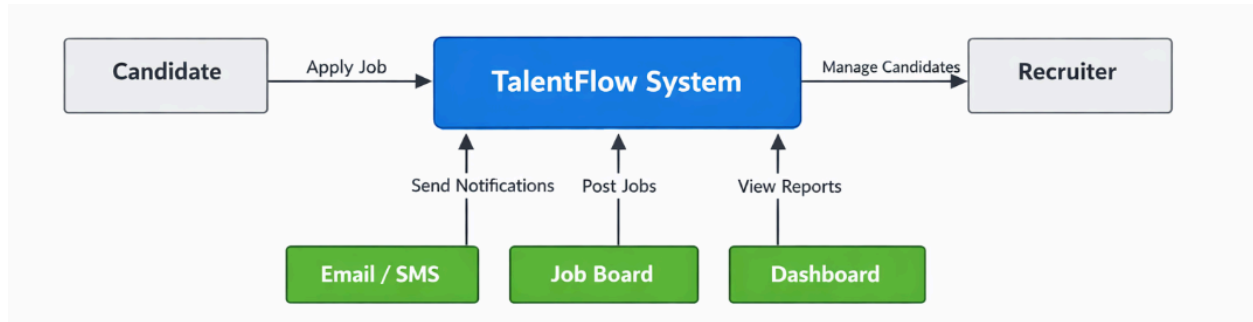


Figure 1

Figure 1 contains a diagram of the TalentFlow System, along with interaction points with other entities. The key users of the TalentFlow System are candidates who apply for the vacancies and the recruiters who process their applications. The TalentFlow System is the key entity that coordinates all these interactions. It also interacts with some external entities to facilitate these processes. These include email/sms notifications for communication, job boards where the recruiter can post their vacant positions, and dashboards for tracking performance.

Component / Service Architecture Diagram

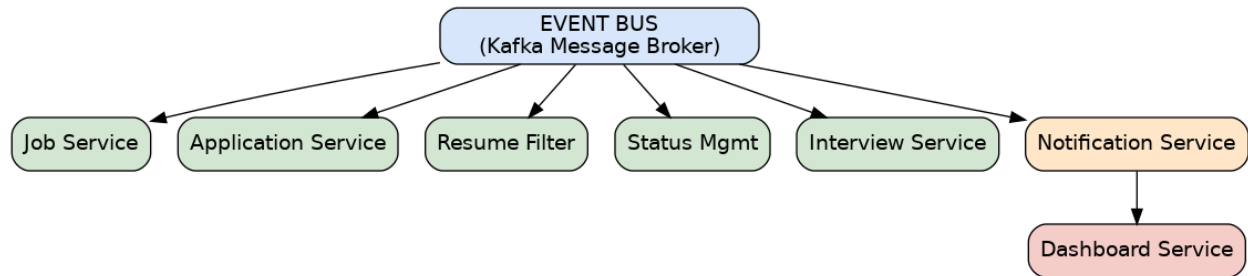


Figure 1.2

Figure 1.2 shows the microservice architecture of the system with the use of the event bus framework based on Apache Kafka. Each main service, namely, the Job Service, Application Service, Resume Filter, Status Management, Interview Service, and Notification Service, communicates using events within the event bus. As a result, there are no direct relationships between services; therefore, loose coupling is achieved. Also, the Notification Service passes events to the Dashboard Service in order to visualize certain data.

Interaction Diagram

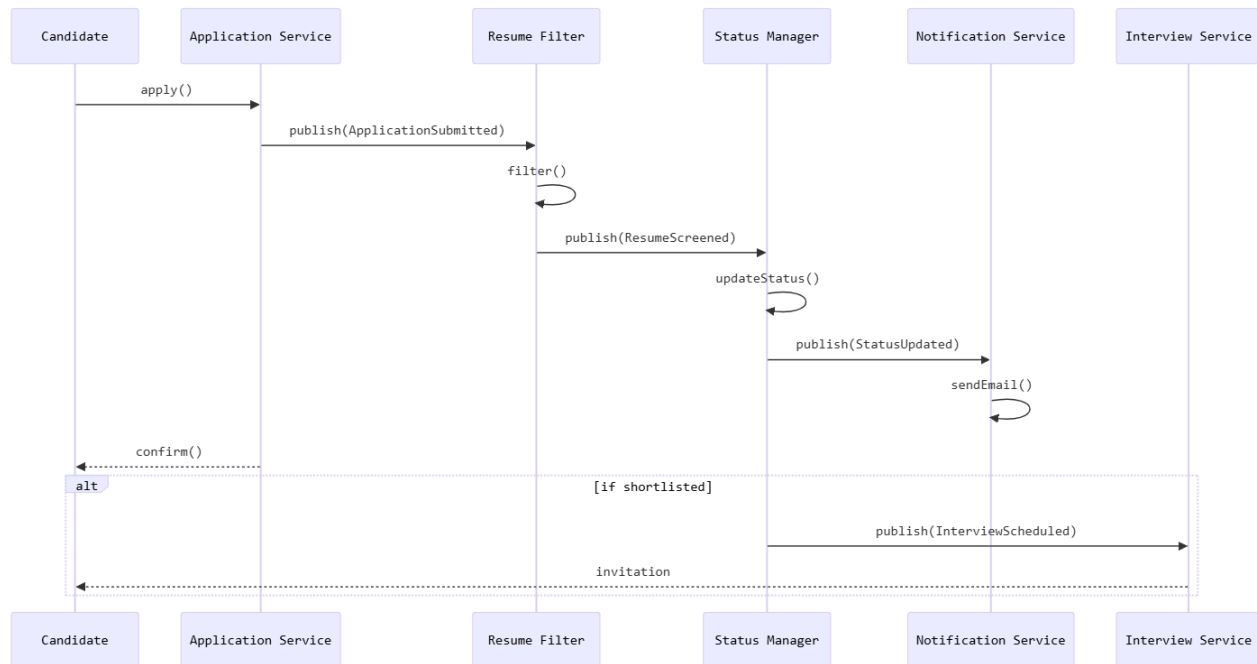


Figure 1.3

Figure 1.3 shows the complete flow of how the job application process works. The process starts with the applicant sending their application via the Application Service, which then triggers the event Application Submitted. This event will be handled by the Resume Filter service, whereby the applicant's resume will be reviewed. Thereafter, the event Resume Screened will be fired, with the Status Manager service updating the status of the applicant's application. An event Status Updated will follow suit, leading to the Notification Service sending an email to the applicant. If the applicant's resume qualifies, there will be an additional flow where an event Interview Scheduled will be triggered, and the applicant will receive an invite for an interview.

Deployment Diagram

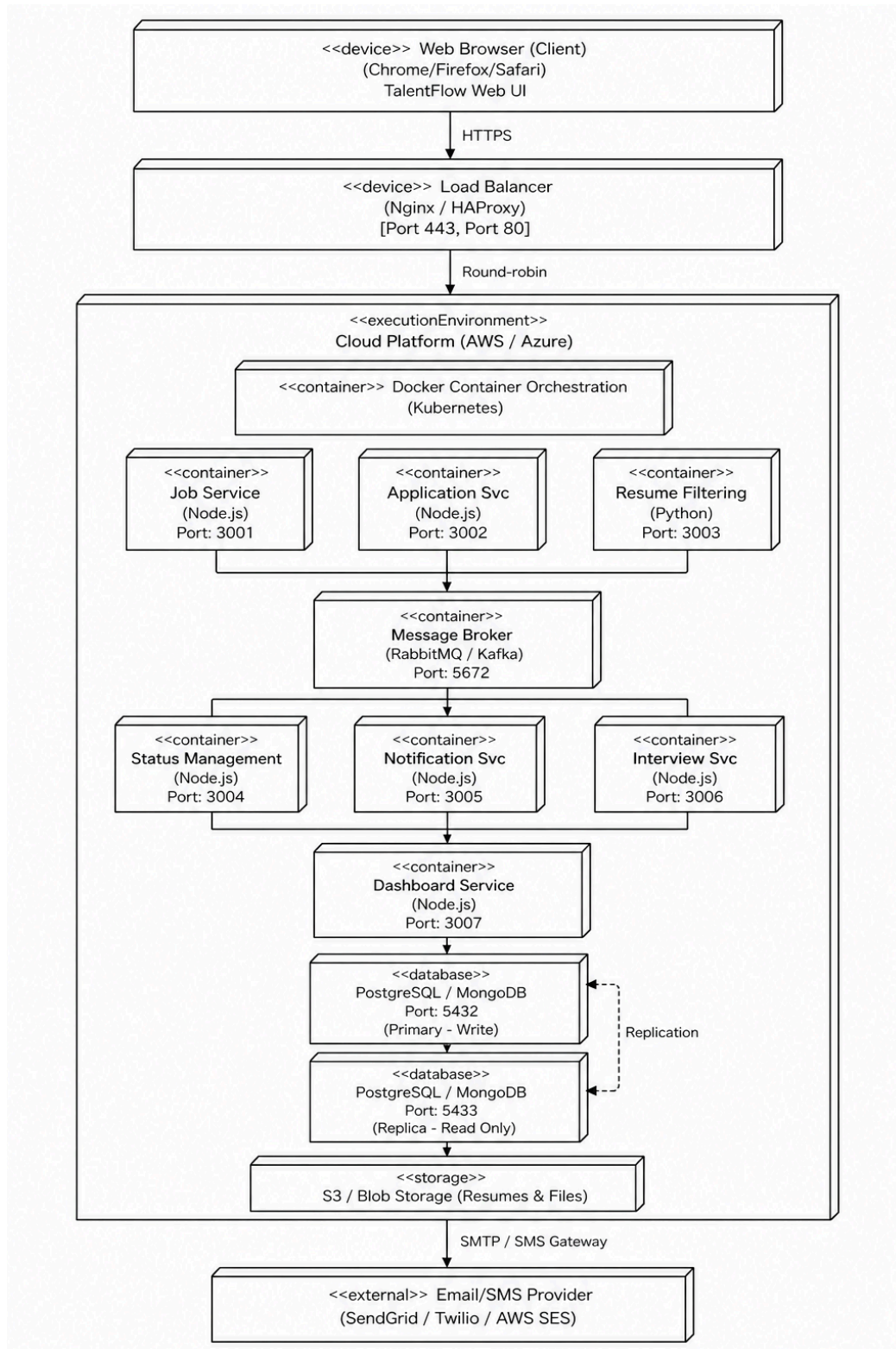


Figure 1.4

The following illustration 1.4 illustrates the TalentFlow system architecture and demonstrates its internal structure. Users (candidates and recruiters) use the system via a web browser and send their requests via a secure HTTPS connection. A load balancer helps distribute the load effectively even with an intensive workload.

As for the infrastructure, the TalentFlow system is deployed in a cloud and consists of different services managed by Kubernetes. Among core services, we can name such as Job Service, Application Service, and Resume Filtering Service responsible for jobs, applications, and resume filtering, respectively. They communicate using the message broker such as RabbitMQ or Kafka supporting event-driven architecture.

Other services include Notification, Interview, Status Management, and Dashboard used for updates, scheduling, and management of statuses, respectively. Moreover, there is a Primary DB and Replica DB involved. In addition, files, such as resumes, are stored in the cloud storage.

Thus, the described architecture provides for scalability and efficiency of the system with high user activity.

.

Team Member Roles

Name	Role	Responsibilities
Nurul Iffah Sabrina Binti Ahmad Shukri	System Architect & Designer	Design system structure, create UML diagrams, and decide architecture
Izz Dhia Damia Binti Mat Zaidi	Backend & Integration Lead	Develop backend system, manage database, and handle API integration